NATIONAL TEACHERS COLLEGE

SOFTWARE DESIGN & ANALYSIS DOCUMENT

EcoWatt: High-Voltage Energy Dashboard
Aligned with UN Sustainable Development Goal 13

IT Elective 2
Bachelor of Science in Information Technology
Group 5 FinWise

Group Members:

Ani, Moeses Darwin
Delos Santos, Peter John
Gentiroy, Earl Vincent
Rivera, Eunice Jade C.

March 2026

I. Introduction

1.1 Project Overview

EcoWatt: High-Voltage Energy Dashboard is a Windows Forms application developed using C# and .NET to monitor and manage household appliance energy consumption. The system aligns with United Nations Sustainable Development Goal 13 (Climate Action) by helping users understand their electricity usage and reduce unnecessary energy consumption.

The application allows users to log appliance information such as wattage and usage duration. It then calculates the estimated energy consumption and visualizes the data through charts and analytics. By providing transparent insights into energy use, EcoWatt encourages users to adopt sustainable energy practices.

1.2 Problem Statement

Many households lack visibility into the energy consumption of individual appliances. Electricity bills typically provide only total usage, making it difficult to identify which devices contribute the most to energy waste.

Without access to detailed energy monitoring tools, users are unable to make informed decisions to reduce their electricity consumption and environmental impact. EcoWatt addresses this problem by providing a centralized platform that allows users to track, visualize, and analyze appliance energy usage, helping bridge the gap between daily habits and environmental sustainability.

II. Requirements & Analysis

2.1 Functional Requirements

- FR1 – Secure Login System
  The system shall implement a secure login and registration feature with password hashing using SHA256.
- FR2 – Energy Logging
  Users shall be able to add, edit, view, and delete appliance energy logs.
- FR3 – Data Persistence
  The system shall store data in a relational database using Entity Framework Core with SQLite.
- FR4 – Data Visualization
  The system shall display energy consumption data using graphical charts.

- FR5 – JSON Export Feature
  The system shall allow users to export recorded energy logs into a JSON file for backup and reporting.

2.2 Non-Functional Requirements

- NFR1 – UI Responsiveness
  All heavy operations such as database writes and file exports must use async/await to ensure the user interface remains responsive.
- NFR2 – Security
  User passwords must be stored using cryptographic hashing and never in plain text.
- NFR3 – Reliability
  All database and file operations must implement exception handling to prevent application crashes.
- NFR4 – Maintainability
  The application must follow a strict N-Tier Architecture, separating the user interface, business logic, and data access layers.

2.3 Complexity Analysis

To ensure scalability, the core methods in the EnergyService.cs (Business Logic Layer) were analyzed for algorithmic efficiency:

A. Calculation Logic

Algorithm: kWh = (Watts × Hours) / 1000	Time Complexity: O(1) — Constant Time

Rationale: Mathematical operations are executed in a fixed number of steps regardless of the number of database records.

B. Search/Filter Logic

Algorithm: LINQ-based filtering on UserId	Time Complexity: O(n) — Linear Time

Rationale: In a standard SQLite scan without advanced indexing, the system checks n records to filter logs belonging to the current user.

C. Data Export

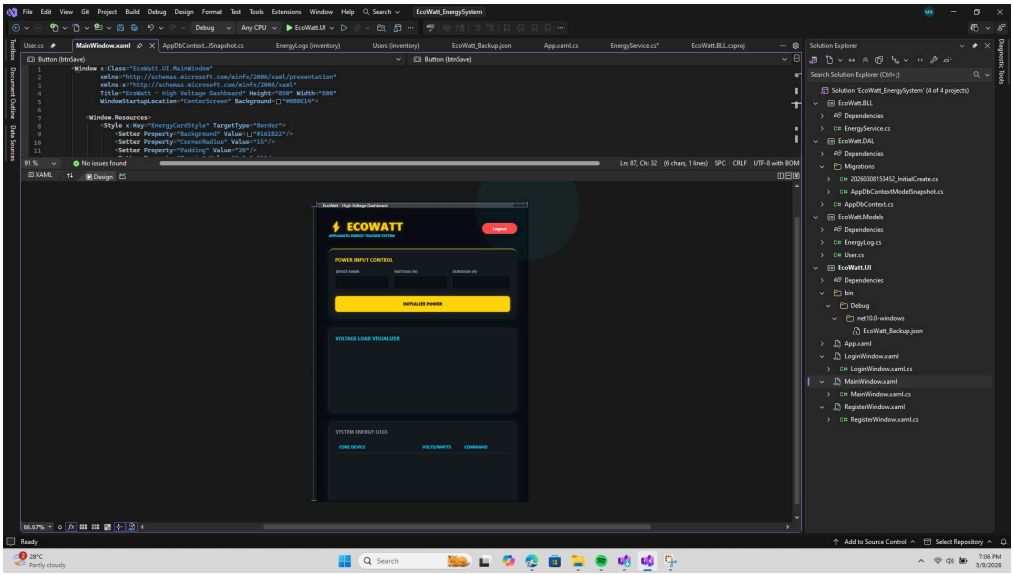
Algorithm: Iterative JSON serialization of log records	Time Complexity: O(n) — Linear Time

Rationale: Each of the n log records must be visited once to convert them into JSON format.

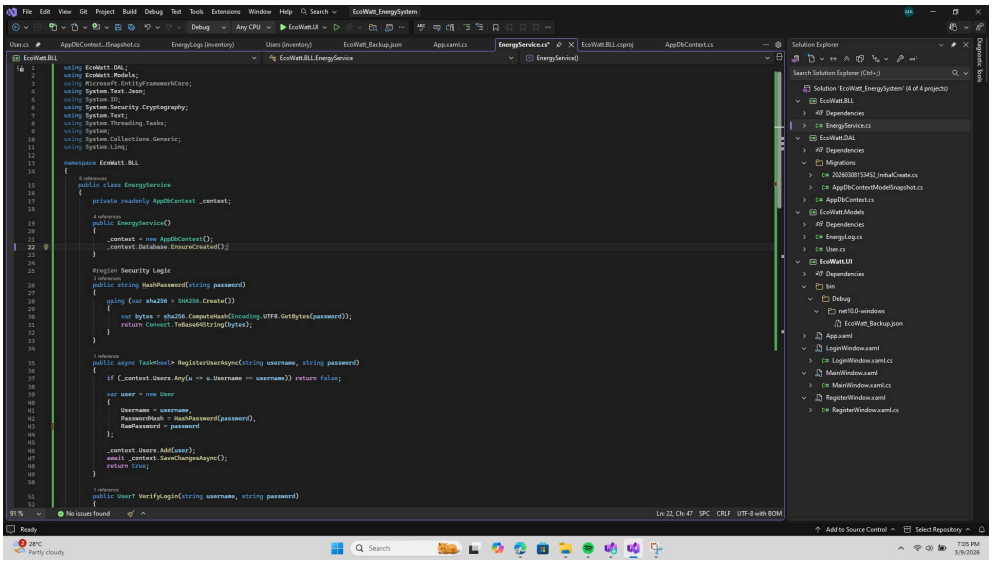
III. Design Specification

3.1 N-Tier Architecture Diagram

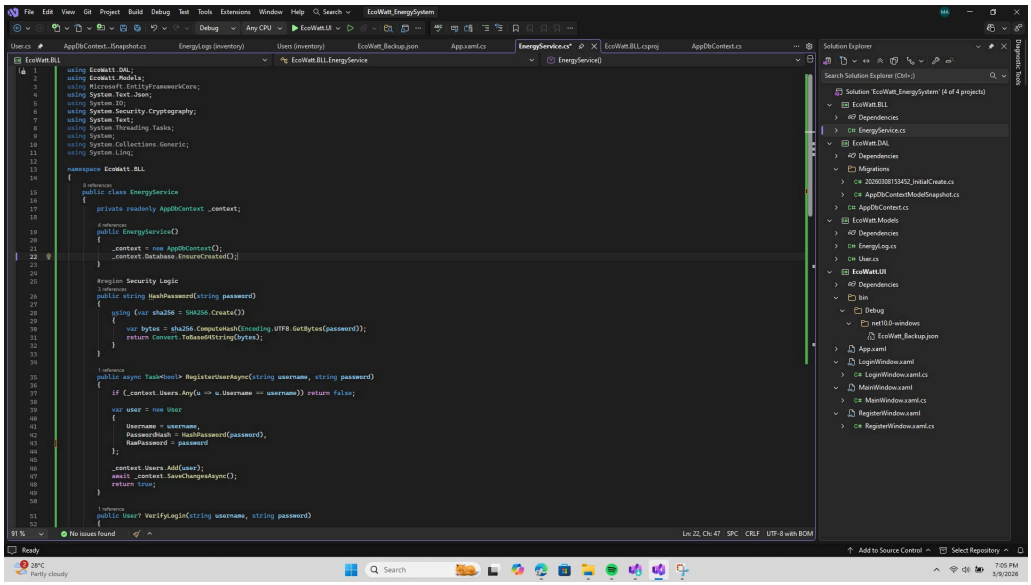
Presentation Layer (UI): WPF (XAML/C#) - MainWindow.xaml



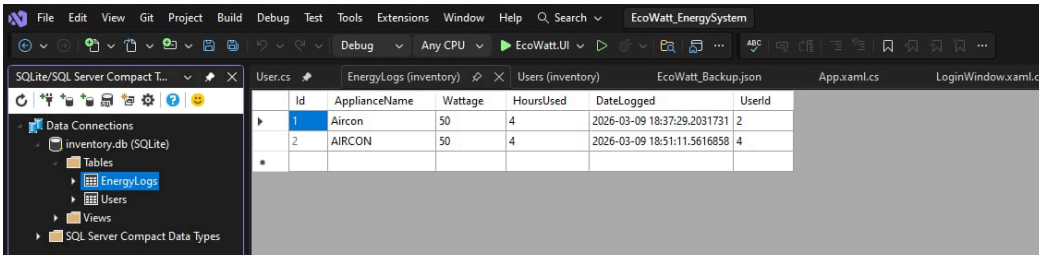
Business Logic Layer (BLL): EnergyService.cs

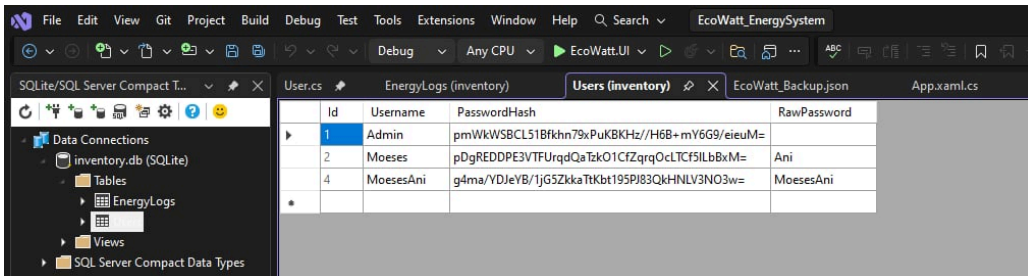


Data Access Layer (DAL): AppDbContext.cs at Repositories

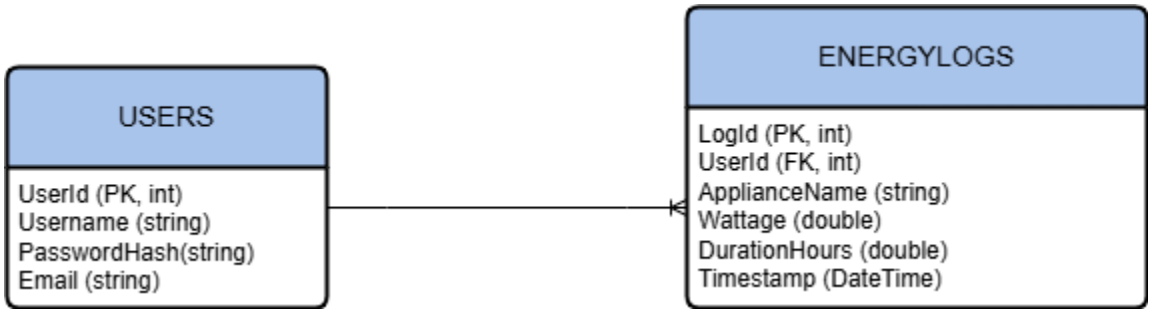


SQLite Database / SQL Database



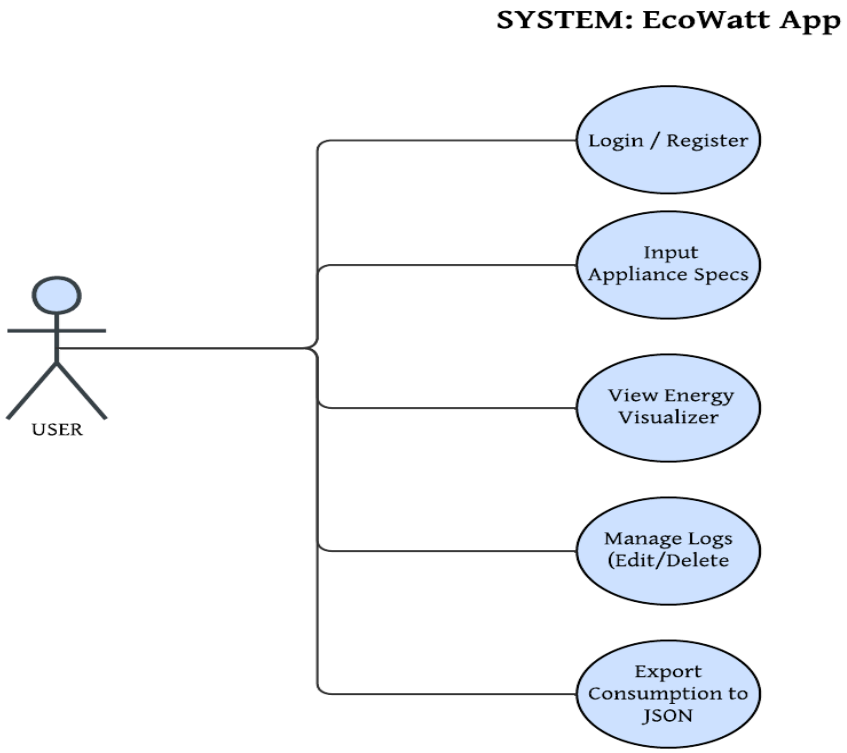


3.2 Entity Relationship Diagram (ERD)



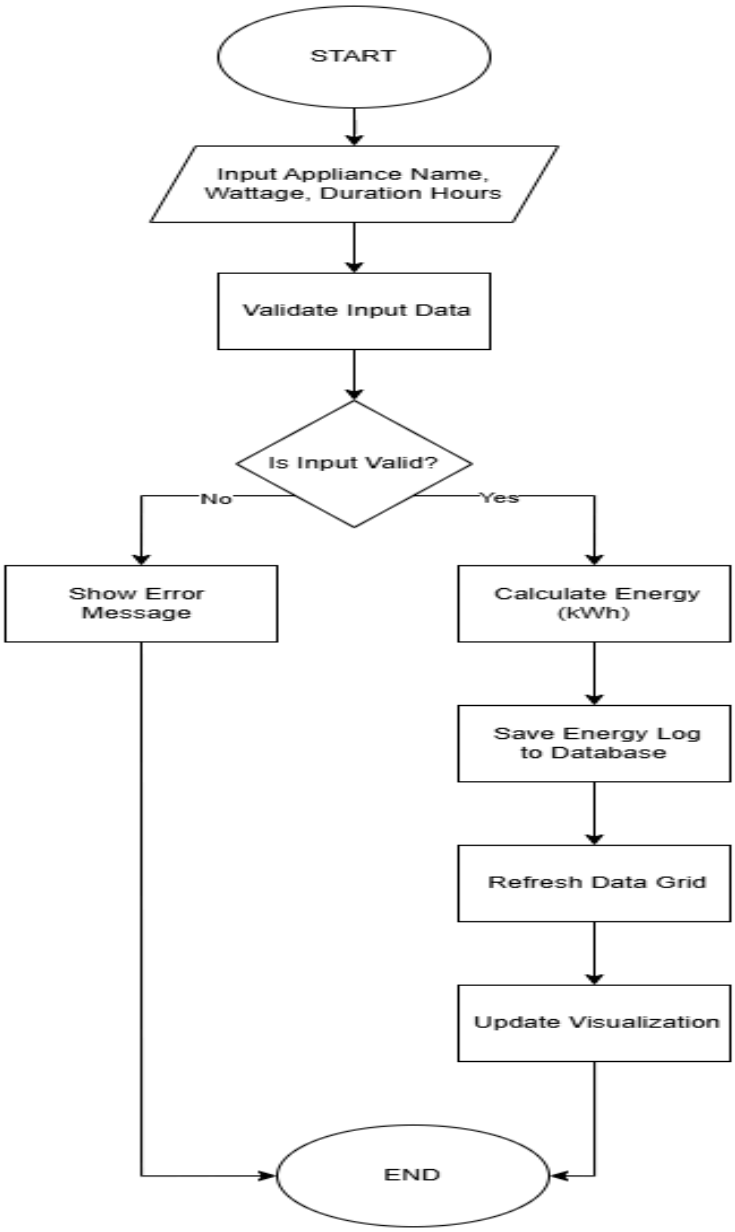
This ERD uses a **one-to-many** relationship to link users to their data. The `Users` table keeps track of accounts using a unique `UserId`, while the `EnergyLogs` table stores individual consumption entries. By using `UserId` as a foreign key in the logs, you ensure that every record is tied to the correct person. It's a clean, standard way to make sure that when a user views their history, they only see their own logs and nothing else.

3.3 Use Case & Data Flow Diagrams (DFD)



This diagram outlines the functional scope of the **EcoWatt** application by mapping out the specific actions a user can perform. The "User" acts as the primary trigger, initiating processes ranging from authentication to data management and export. Each oval represents a distinct system feature, defining exactly what the application does without getting into the technical details of the code or database structure. In short, it visualizes the user’s journey through the app's core capabilities.

3.4 Algorithm Flowchart



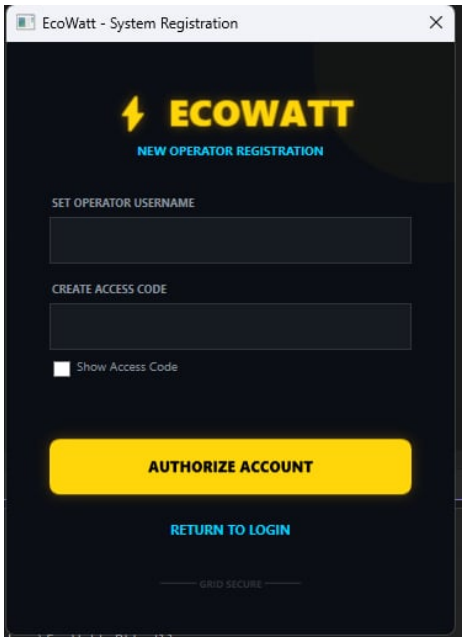
This flowchart maps the btnSave logic. It starts by validating user input; if valid, it computes the energy consumption (kWh) and saves it to the database. Finally, it refreshes the UI components to display the updated data. If input fails, it triggers an error message to prevent invalid data from being saved.

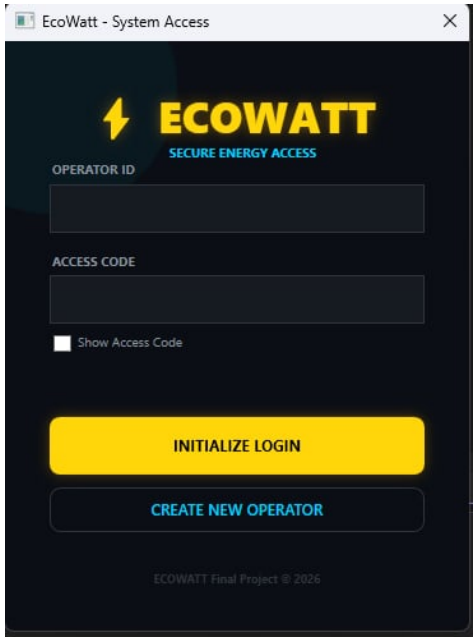
IV. Individual Contributions

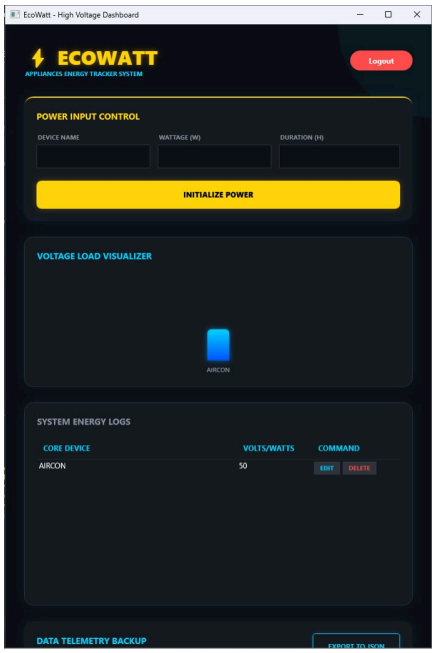
This section outlines the roles and responsibilities of each team member during the development of the EcoWatt system.

Team Member	Assigned Task	Description of Contribution
Ani, Moeses, Darwin	System Development (Coding)	Responsible for implementing the core functionalities of the system including program structure, operations, business logic implementation, and system testing.
Delos Santos, Peter John	Project Support	Assisted in project coordination and preparation of some project requirements during development.
Gentiroy, Earl Vincent	System Design & Diagrams	Responsible for creating system diagrams such as ERD, editing and graphical design components.
Rivera, Eunice Jade C.	Documentation & Finalization	Prepared the SDAD documentation, presentation script, formatting of diagrams, and handled the overall finalization of project requirements.

V. Appendices







EcoWatt is an energy monitoring application developed using C# and .NET that helps users track the electricity consumption of household appliances. Users can input appliance details such as wattage and usage time to calculate energy consumption in kilowatt-hours (kWh). The system also displays charts and visualizations to help users understand their energy usage. EcoWatt promotes energy awareness and supports sustainable practices aligned with SDG 13